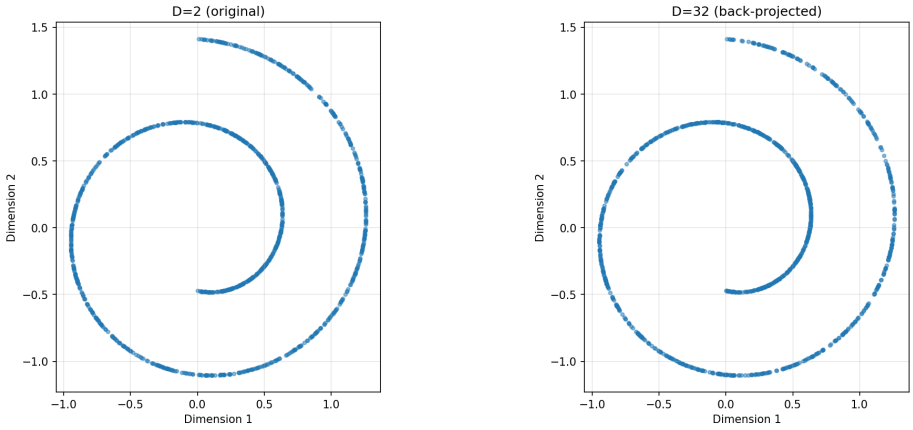
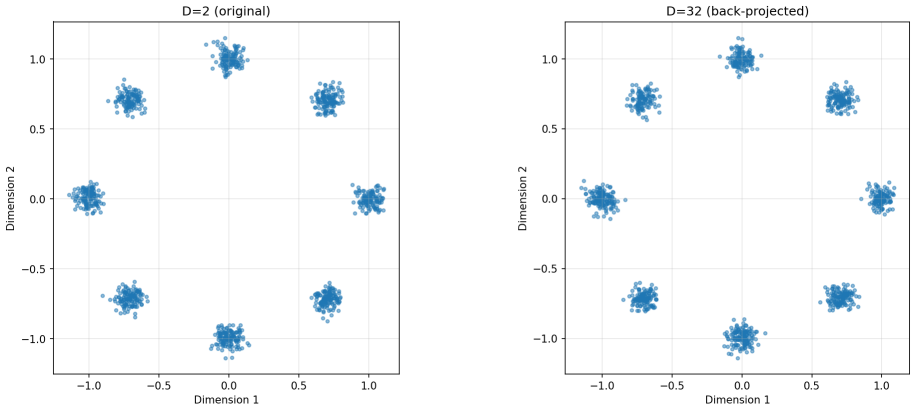
Part 1

Data Visualisation

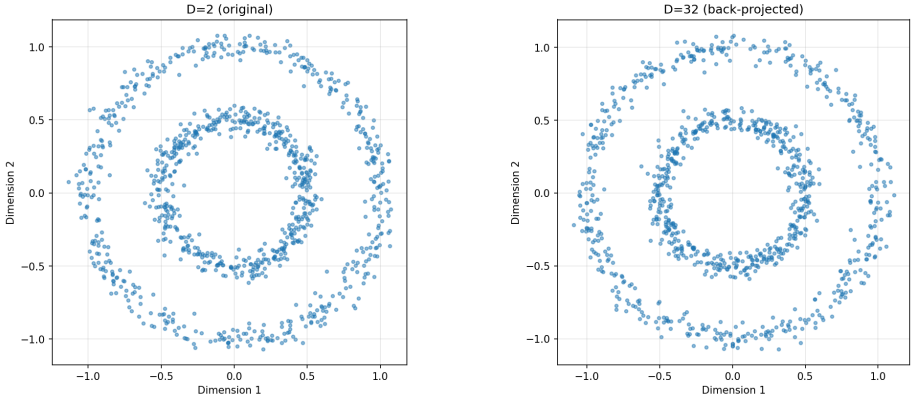
swiss_roll



gaussians



circles



v-Prediction Flow Matching at `D = 2`

Hyperparameters:

n_steps	lr	batch_size	steps	n_gen	clamp
25000	1e-3	1024	50	2048	0.01

Trained on v-prediction (target) with v-loss

Part 2

Calculate flow matching based on x or v-pred

We know (1) $z_t = (1-t)x + t\epsilon$. Further it is also defined that (2) $v = \epsilon - x$.

Rearranging (2) we get $\epsilon = v + x$, substituing this back into (1),

$$z_t = (1-t)x + t(v+x)$$

$$z_t = x - tx + tv + tx$$

$$z_t = x + tv \Rightarrow \boxed{x = z_t - tv} \Rightarrow \boxed{v = \frac{z_t - x}{t}}$$

Hence in the scenario we predict x with a loss in v, we can use the above equation to determine:

$$v_{\text{output}} = \frac{z_t - x_{\text{pred}}}{t} \qquad v_{\text{target}} = \epsilon - x$$

Similarly, for predicting v with a loss in x, we have:

$$x_{\text{output}} = z_t - tv_{\text{pred}} \qquad x_{\text{target}} = x$$

Reproduce papers findings

We use the same hyperparameters as Part 1.2.

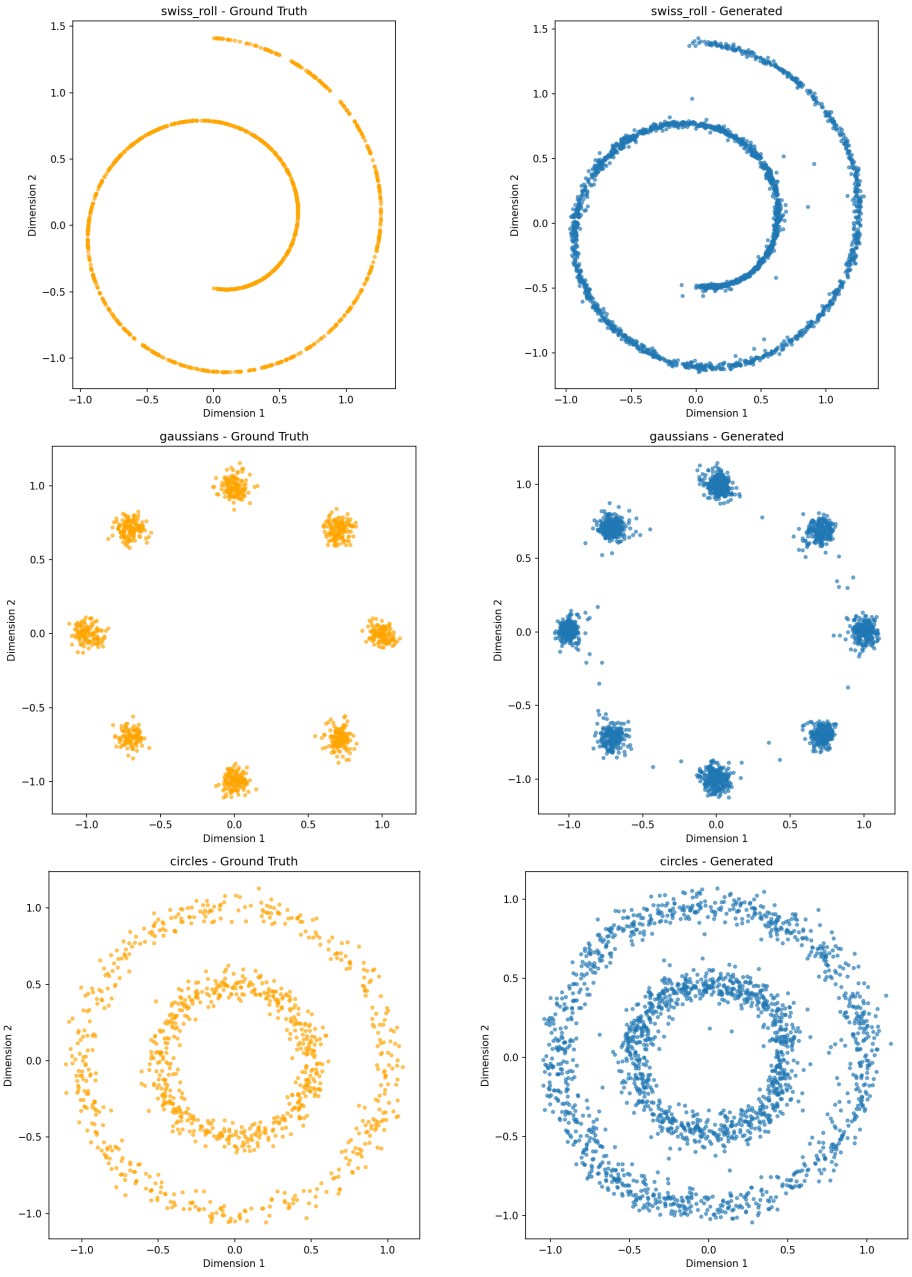
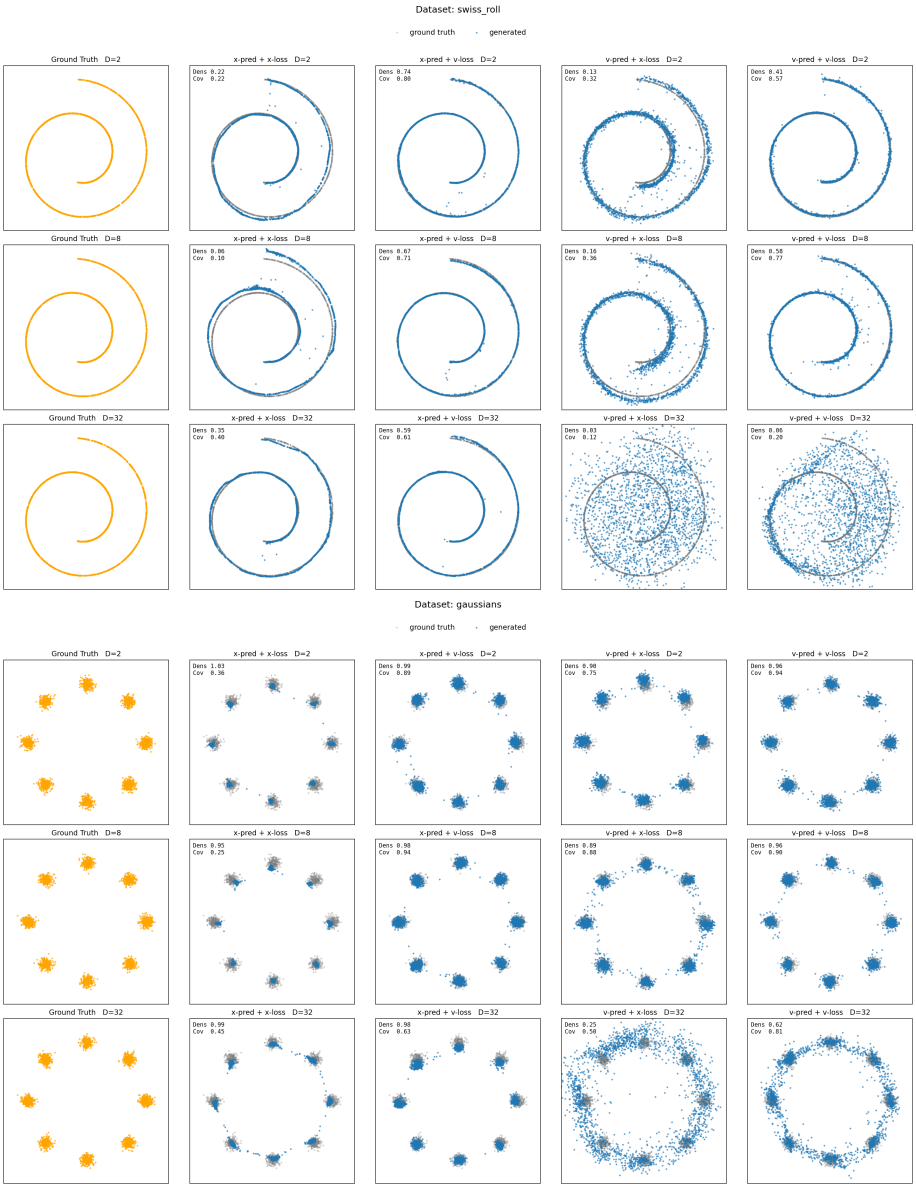
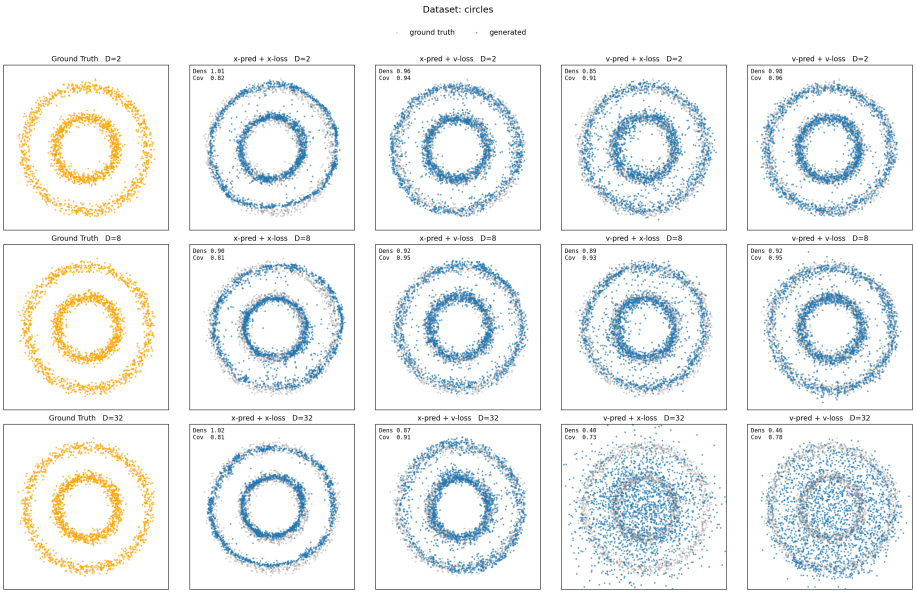


Figure 1: Grid


Dataset: swiss_roll
ground truth
generated
Ground Truth D=2
x-pred + x-loss D=2
Dens 0.22
Cov 0.22
x-pred + v-loss D=2
Dens 0.74
Cov 0.80
v-pred + x-loss D=2
Dens 0.13
Cov 0.32
v-pred + v-loss D=2
Dens 0.41
Cov 0.57
Ground Truth D=8
x-pred + x-loss D=8
Dens 0.06
Cov 0.10
x-pred + v-loss D=8
Dens 0.67
Cov 0.71
v-pred + x-loss D=8
Dens 0.16
Cov 0.36
v-pred + v-loss D=8
Dens 0.58
Cov 0.77
Ground Truth D=32
x-pred + x-loss D=32
Dens 0.35
Cov 0.40
x-pred + v-loss D=32
Dens 0.59
Cov 0.61
v-pred + x-loss D=32
Dens 0.03
Cov 0.12
v-pred + v-loss D=32
Dens 0.06
Cov 0.20
Dataset: gaussians
ground truth
generated
Ground Truth D=2
x-pred + x-loss D=2
Dens 1.03
Cov 0.36
x-pred + v-loss D=2
Dens 0.99
Cov 0.89
v-pred + x-loss D=2
Dens 0.90
Cov 0.75
v-pred + v-loss D=2
Dens 0.96
Cov 0.94
Ground Truth D=8
x-pred + x-loss D=8
Dens 0.95
Cov 0.25
x-pred + v-loss D=8
Dens 0.98
Cov 0.94
v-pred + x-loss D=8
Dens 0.89
Cov 0.88
v-pred + v-loss D=8
Dens 0.96
Cov 0.90
Ground Truth D=32
x-pred + x-loss D=32
Dens 0.99
Cov 0.45
x-pred + v-loss D=32
Dens 0.98
Cov 0.63
v-pred + x-loss D=32
Dens 0.25
Cov 0.50
v-pred + v-loss D=32
Dens 0.62
Cov 0.81



Q1. Prediction-type scaling

It is clear that x-prediction scales to $D = 32$ better than v-prediction, across the three datasets. Qualitatively $D = 32$ with x-pred least resembles the target dataset. To make our analysis more robust, we measure generation density & coverage [1], a strong metric for comparing generation with ground truth.

> **Density** is average number of real-point k-NN balls each generated sample falls into, divided by k [1]
>
> **Coverage** is fraction of real points whose k-NN ball contains at least one generated sample (k=5, ball radius = distance to k-th real neighbour). [1]

Below we plot density/coverage as a function of the ambient dimention D:

The trend is clear, v-pred (green and red) scales the worse than x-pred (blue and orange). This is consistent with our generated results.

$$x_{\text{pred}} >_{\text{dim scaling}} v_{\text{pred}}$$

Q2. Loss-space

The impact of loss space appears to depend on the prediction type. When inspecting the density/converage graph above, the v-pred appears to follow the exact same trend, with a slight offset. x-pred on the other hand displays similar scaling (over D) across the loss types, but the offset is signficant, with the blue (x-loss) being much lower than the orange (v-loss).

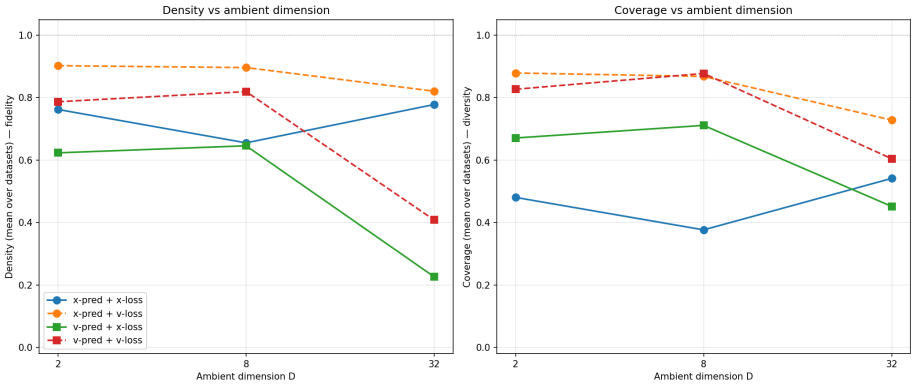


Figure 2: density_coverage_vs_dimension

These observations are convincingly backed by the following normalised loss plot (where loss is scaled by starting value):

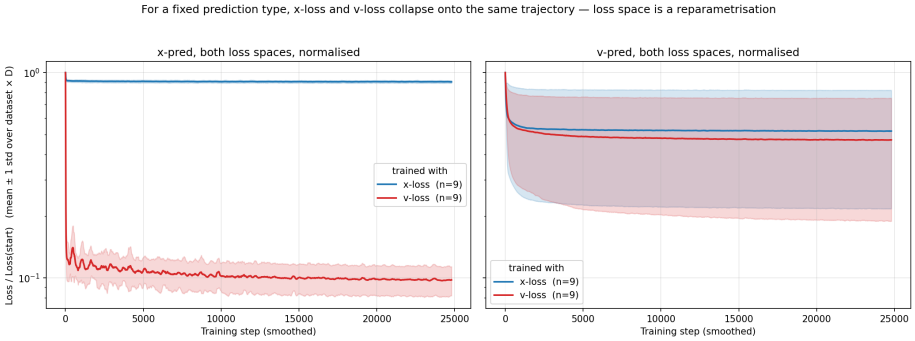


Figure 3: loss_curves_normalised

The plots demonstrate that loss has minor impact on convergence for v-pred, while v-loss is significantly stronger for x-pred.

This being said, when observing the density/coverage figure, the dominating factor behind performance is certainly the prediction type, with loss having a minor impact in the x-pred scenario.

Q3. Why does x-pred scale to higher D

x-prediction scales superior to v-prediction as D grows, as the signal to be learned is better preserved. For x-pred at a high D, the target data lies on a 2D manifold under R^D (well described in [2]). Therefore the model is intrinsically learning in 2D, regardless of D, i.e., a projection onto the manifold. I.e., $rank(x)_{\text{intrinsic}} = 2$.

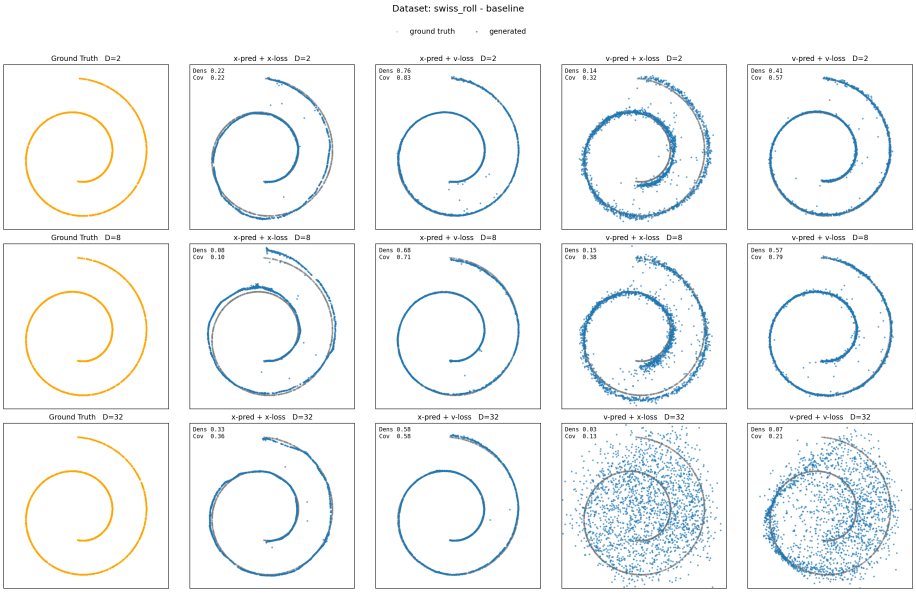
Meanwhile, for v-prediction, the model is tasked with learning $v = \epsilon - x$. Unlike x, ϵ is full-rank Guassian noise in R^D (i.e., intrinsic dim $= D$). This implies v measurments are dominated by ϵ, meaning $\uparrow D$ results in linearly increased complexity in v, while x's complexity remains at 2. From inductive bias, it follows that the model will struggle to fit the taret.

As for the loss-space, the MSE is a t-dependent reweighting, meaning the underlying objective being learned remains the same.

Part 3: Can We Rescue v-Prediction?

From Part 2 we established that v-prediction degrades sharply as D grows, with $D \geq 32$ giving qualitatively poor samples on the `swiss_roll` dataset. The diagnosis there was that $v = \epsilon - x$ inherits full-rank Gaussian noise in $\mathbb{R}^D$, so the target's intrinsic dimensionality scales with D while x's stays at 2. This part tests whether that failure is fundamental or an artefact of the noise process being mismatched with the data manifold.

The Part 2 baseline on `swiss_roll` reproduced below for reference:



Visualisations: what works and what doesn't

`opt001`: Noise dimension matched to data manifold.

Relating to our hypothesis from part 2, [2] outlines a key insight, **the process should lie on the same manifold as the data**. From this the we have our first proposition **reduce the intrinic dimenionality of noise to match data, independent of** D.

To achieve this, it is natural to use the randomly initalised matrix P, responsible for projecting $R^D \to R^2$. We outline the process as follows,

1. Sample $\epsilon_2 \sim \mathcal{N}(0, I)$, where $\epsilon_2 \in R^2$
2. Project to R^D: $\epsilon_D = \epsilon_2 \cdot P$, where $\epsilon_D \in R^D$ and $P \in R^{2\times D}$

The same procedure must be applied at sampling time, since the model has only ever seen noise on the data subspace. Concretely, the initial state of the ODE is drawn as $z_1 = \epsilon_2 \cdot P$ rather than $z_1 \sim \mathcal{N}(0, I_D)$, sampling from full-rank Gaussian noise in $\mathbb{R}^D$ would push the trajectory off the data manifold and break the train/test consistency that this rescue depends on. Below are results using this noise dim optimisation:

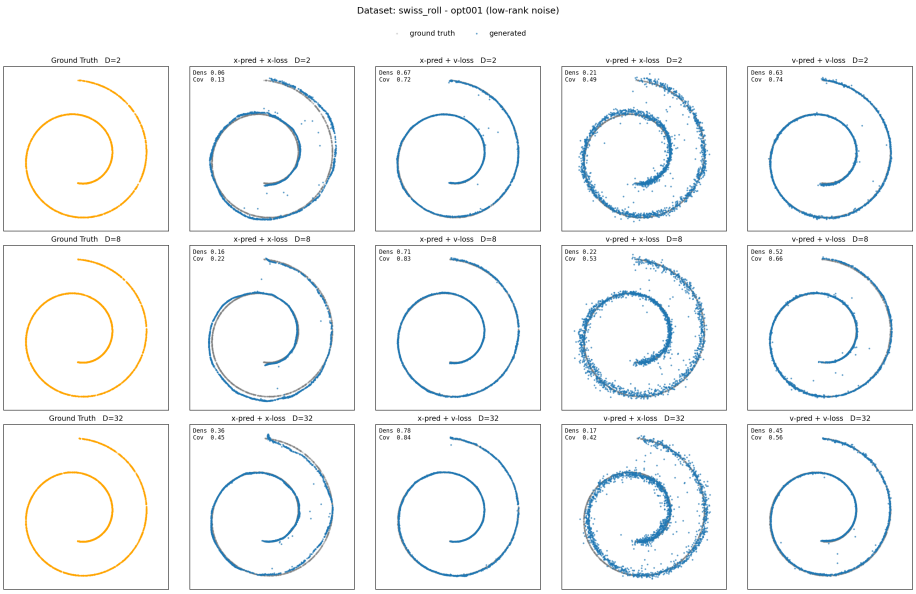


Figure 4: swiss_roll_grid_opt001

`opt002`: Width scaling on top of `opt001`.

The effect on v-pred at $D = 32$ is dramatic, samples recover recognisable swiss-roll structure where the baseline produced a diffuse blob. x-pred is essentially unchanged.

Section 4 of [3] argues wider networks specifically help v-prediction. We re-ran with $h = 512$ and $h = 1024$:

Width gives an additional but smaller bump, v-pred at $D = 32$ tracks the ground-truth shape a bit more tightly, though $h = 512$ vs $h = 1024$ is hard to separate qualitatively. The big win is `opt001`; `opt002` is incremental.

Summary of what works/doesn't: matching the noise process to the data

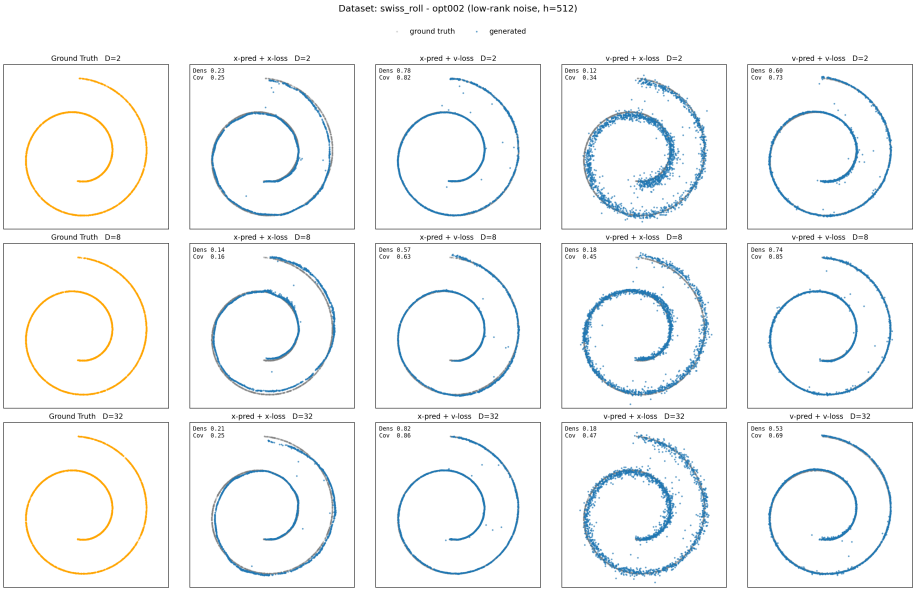


Figure 5: swiss_roll_grid_opt002_h512

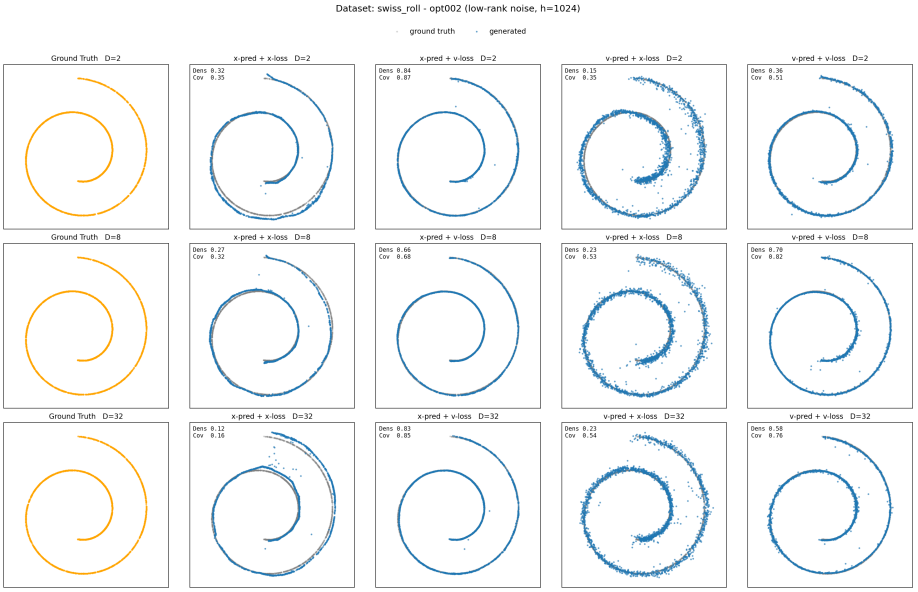


Figure 6: swiss_roll_grid_opt002_h1024

manifold (`opt001`) is the decisive intervention and rescues v-pred at $D = 32$ at zero additional parameter cost. Width scaling helps further but with steeply diminishing returns. x-pred is unaffected by either change because it was already working, its target is intrinsically 2D regardless of D.

Q1. Is v-prediction's failure fundamental?

No, the failure is parameterisation-dependent, not fundamental. Once the noise process is restricted to the same 2D subspace as the data, v-pred at $D = 32$ produces samples comparable in shape to x-pred (visually obvious in the `opt001` grid above; quantitatively the v-loss with v-pred drops from 0.0716 at baseline to 0.0587 with `opt001`, despite identical architecture and parameter count, see Table 1 in appendix for more detail).

This **supports** the Part 2 analysis rather than contradicting it. In Part 2 we attributed the scaling gap to v's dependence on full-rank noise: $v = \epsilon - x$ has intrinsic rank that grows with D, while x stays on a 2D manifold. The rescue here works *exactly* by removing that asymmetry, projecting ϵ onto the data manifold so v's intrinsic dimensionality also stays at 2. The fact that this single change closes most of the gap is direct evidence that full-rank noise was the root cause we identified in Part 2, not some inherent property of the v parameterisation.

Q2. Approaches tried and compute cost vs. default x-prediction

Two interventions, applied cumulatively: 1. **Manifold-aligned noise (`opt001`)**: sample $\epsilon \in \mathbb{R}^2$ and project to $\mathbb{R}^D$ via the same P used for the data, at both training and sampling time. No architectural change. 2. **Width scaling (`opt002`)**, increase hidden size from $h = 256$ to $h \in \{512, 1024\}$ on top of `opt001`.

config	params	rel. params	time/run	final loss (v-loss)
default x-pred (baseline)	312,608	1.0×	48s	0.0165 (x-loss)
baseline v-pred	312,608	1.0×	46s	0.0716
`opt001` v-pred	312,608	**1.0×**	47s	0.0587
`opt002_h512` v-pred	1,149,472	3.7×	48s	0.0589
`opt002_h1024` v-pred	4,396,064	14.1×	75s	0.0588

An intresting observation is that `opt001`'s increased performance comes at no increased compute cost.

Quanitfying performance via loss (especially when x-pred calculated using a different loss althougher) is uninformative. When analysing the density/coverage

graph (see appendix table), we see `opt001` v-pred lands close to default x-pred, and `opt002` only marginally further.

So to match default x-pred sample quality at $D = 32$, v-pred needs `opt001` at minimum (free) and benefits modestly from `opt002_h512` (3.7× params for a small additional gain). x-pred remains the cheaper route in absolute terms, but the gap is far smaller than Part 2 suggested it would be.

Q3. How do x-pred and v-pred respond to these changes?

They respond very differently. The figure below plots density/coverage averaged over D, with the optimisation stage on the x-axis, one panel per (pred, loss) combination:

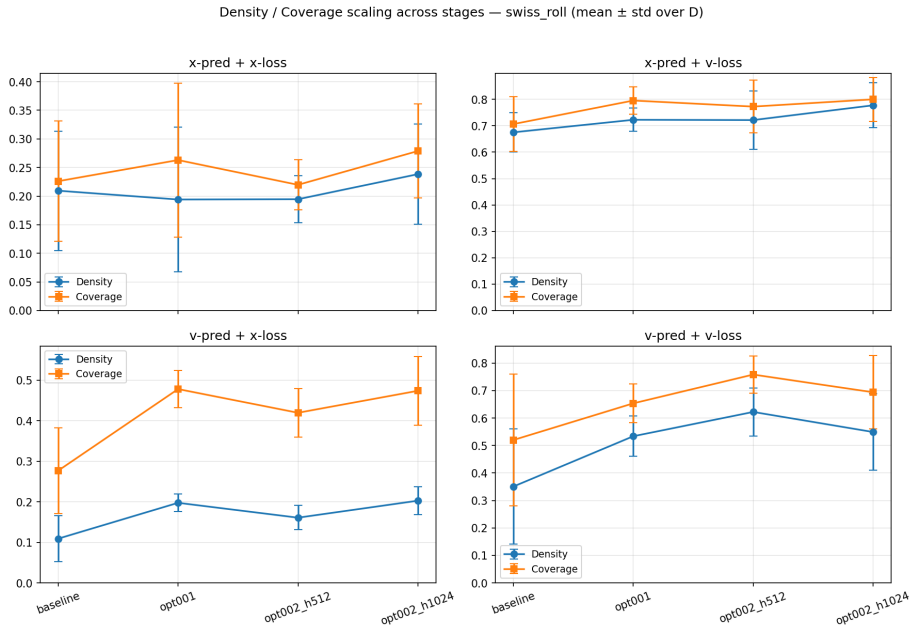


Figure 7: scaling_dens_cov_per_predloss

x-pred (top row) is approximately flat across `baseline -> opt001 -> opt002_h512 -> opt002_h1024`, both density and coverage barely move. v-pred (bottom row) shows a large `baseline -> opt001` jump, then near-flat `opt001 -> opt002` behaviour. The two parameterisations have completely different sensitivity profiles to the same set of changes.

The reason follows from what each parameterization has to learn as a function of D:

- **x-pred's target is intrinsically 2D regardless of D.** The clean data lies on the 2D manifold $\{xP : x \in \mathbb{R}^2\} \subset \mathbb{R}^D$, and that's true at $D = 2$ and

at $D = 32$. So x-pred is already aligned with the manifold by construction, projecting noise onto the manifold doesn't change what's being learned, and adding width doesn't help fit a target that was already simple. There's nothing for these changes to fix.

- **v-pred's target inherits the dimensionality of ϵ.** With full-rank Gaussian noise in $\mathbb{R}^D$, $v = \epsilon - x$ has intrinsic dimensionality D, so the model is fitting a D-dimensional regression target even though the data is 2D. `opt001` directly attacks this, projecting ϵ onto the manifold collapses v's intrinsic rank back to 2, restoring symmetry with x-pred. That's why the baseline opt001 jump is large. Once the target is again 2D, extra width (`opt002`) is fitting an already-tractable signal, so its marginal benefit is small, exactly the same regime x-pred has been in all along.

In short, x-pred and v-pred converge to the same response curve once the noise dimensionality is fixed. The asymmetry observed in Part 2 wasn't a property of the parameterisations themselves, it was a consequence of the noise process being mismatched with the data manifold for v but not for x.

Q4. Why does v-prediction behave differently in real image gen systems like FLUX and SD3

SD3 and FLUX are fundamentally different as they do not operate on pixels, instead they operate in an image latent space. The general process of [4] and [5] is as follows: they start with noise in this latent space (conditioned on text), perform diffusion in that space, and pass the final output through a decoding autoencoder to produce RGB images.

This has a significant impact on our v-pred ambient vs. intrinsic D argument. VAEs in SD3/FLUX are trained to make the latent space efficient, where reconstruction quality is the upper bound on generation quality. This means the autoencoder is pushed to capacity, in other words, the autoencoder is essentially doing our `opt001` for free, but learned.

Another reason v-prediction works well in these models is a timestep optimisation in SD3. Paper [4] observes that v-pred targets are trivial at the endpoints (at $t = 0$ or $t = 1$, one of ϵ or x is fully known, so the optimal prediction collapses to a distribution mean) and become harder to predict in between, where the actual learning signal lives. They exploit this with a *logit-normal* timestep sampler that biases training toward intermediate t, focusing compute where the signal is hardest.

Other minor reasons for the situation being different: - **Scale:** the increased `hidden_layers` improvement is naturally present in these models, due to their size. - **Conditioning:** these models are conditioned, implying better learning signals.

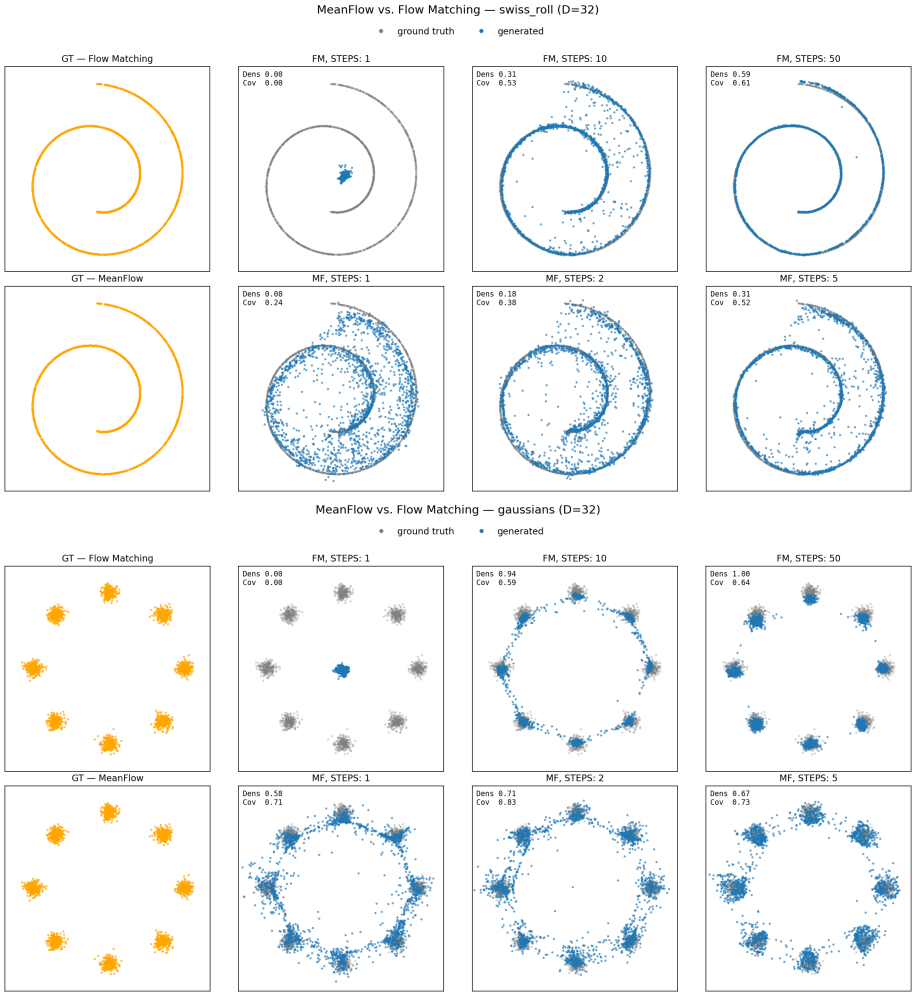
Part 4

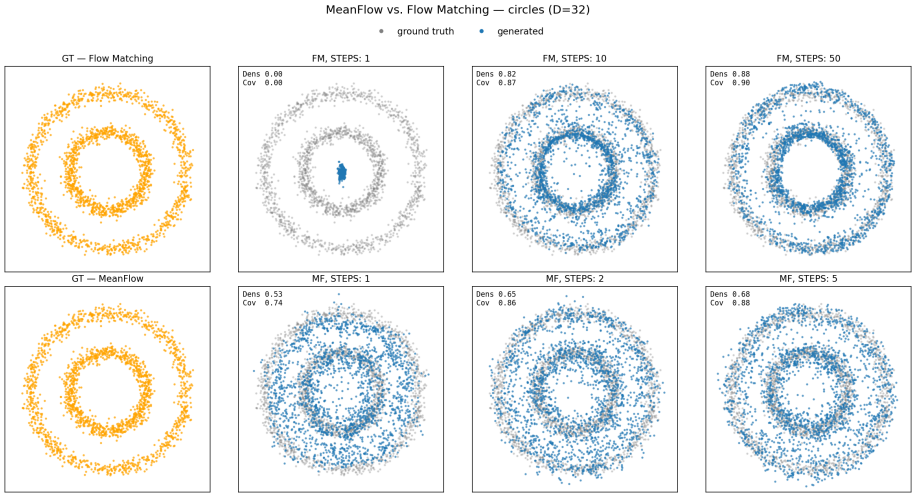
Q1. Sampling Efficiency

From our density/coverage graphs, the best model from part 2) was x-pred/v-loss at D=2. Below we run this model across a range of sampling steps across all the datasets:

Q2. MeanFlow

We produced the following at `mf_ratio=0.5` and $D = 32$ with steps [1,2,5] for MeanFlow and [1,10,50] from our part 2 model (x-pred/v-pred for both). All other hyperparameters remain the same as part 2 in part 4):





Prediction Type for MeanFlow?

We use x-prediction for the MeanFlow MLP. This was the parameterisation that performed well at $D = 32$ in Part 2, and the reason carries directly into Part 4.

The argument from Part 2: x-pred's target is the clean data, which lives on the 2D data manifold regardless of how large the ambient D is. So the *intrinsic* dimensionality of what the model has to learn stays at 2. v-pred's target is $v = \epsilon - x$, which inherits the dimensionality of ϵ, full-rank Gaussian in $\mathbb{R}^D$, so the regression target's intrinsic dim grows with D. At $D = 32$ the density/coverage gap between the two parameterisations was large.

Part 4 trains at $D = 32$, so picking x-pred lets MeanFlow inherit the same favourable scaling. See Appendix MeanFlow Implementation for details on how x-prediction was achieved and how loss is calculated.

Core Idea behind MeanFlow?

Standard flow matching learns the **instantaneous velocity** $v(z_t, t)$, the tangent of the trajectory at a single point in time. To actually sample, you integrate this ODE numerically: take a small Euler step, re-evaluate v at the new z, step again, and so on. You need many steps because v changes along the trajectory, each Euler step is a linear approximation, where big steps might miss the curve entirely. The figure from the MeanFlow paper [6] shows exactly this: one large Euler step shoots off the true path.

MeanFlow's core idea is to learn the **average velocity** over an interval $[r, t]$ instead:

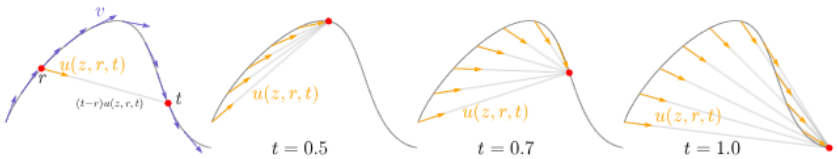


Figure 3: **The field of *average velocity*** $u(z, r, t)$. **Leftmost**: While the *instantaneous velocity* v determines the tangent direction of the path, the average velocity $u(z, r, t)$, defined in Eq. (3), is generally not aligned with v. The average velocity is aligned with the *displacement*, which is $(t - r)u(z, r, t)$. **Right three subplots**: The field $u(z, r, t)$ is conditioned on both r and t, and is shown here for $t = 0.5$, 0.7, and 1.0.

Figure 8: mf_paper

$$u(z_t, r, t) = \frac{1}{t - r} \int_r^t v(z_s, s)\, ds$$

By construction, the displacement over $[r, t]$ equals exactly $(t - r) \cdot u$, therefore no Euler approximation or accumulated error. So one step from $t = 1$ to $r = 0$ using $u(z_1, 0, 1)$ lands at the data point in a single forward pass.

What does the model learn that's different? Standard FM's velocity field is a function of two variables (z_t, t), it only encodes local tangent information. MeanFlow's mean velocity is a function of three: (z_t, r, t). The extra input $h = t - r$ tells the model how far ahead it needs to "look", so it has to encode the *global shape* of the trajectory over a finite interval, not just the instantaneous direction.

Training pays for this. Since you can't compute the integral directly from one sample, MeanFlow uses a self-consistency identity $u = v - h \cdot du/dt$ that the model must satisfy. The JVP is what computes du/dt.

This is what enables one-step generation: at sampling time, a single forward pass outputs the exact mean velocity for the full $[0, 1]$ interval. Thereafter the displacement formula $z_0 = z_1 - u$ finishes the job.

Why is the MeanFlow training split required?

MeanFlows target is `u_tgt_mf = v - h* dudt`, unlike a direct regression (as in standard FM) this quantity contains $\frac{du}{dt}$, i.e., the right-hand side (target) contains the model's own time derivative. Therefore model is told to "match" a quantity that depends on itself. This caveat means there exist many solutions (like PDEs) including many non-optimal.

Using $h = 0$ for a portion of the time, results in the $\frac{du}{dt}$ terms being ignored (i.e., training like standard flow matching). In this time, the model trains with no

self-reference, anchoring the model to a more optimal part of the solution space. After sufficent training, $h > 0$ can be set with the model remaining anchored by the initial training.

Compare training cost to standard flow matching, what is overhead of JVP operation?

The notable increase in cost is the JVP operation. From the experimental data (table 2 in appendix), we notice that training time is roughtly 1.97x greater for the MeanFlow variant, model size and training parameters. The reason for this is what the JP (Jacobian-vector product) is calculating. For a function $f(x)$ and tangent $\dot{x}$, it returns $(f(x), \nabla f(x) \cdot \dot{x})$, which is roughtly double the work for each operations. This also means that PyTorch's autodiff needs to traverse a roughtly 2x larger graph for backpropogation. End-to-end (since this is dominating complexity), this results in 2x longer training steps.

Compare the MeanFlow-generated samples against the ground truth

Qualitative Observation: From a visual standpoint, the MeanFlow generations appears to have great coverage over the target, but noise (which costs in the density department). One step MF signficantly outpeforms one step FM as expected. Beyond this, the noise appears to be in the form of collaposed points (i.e., between two possible target locations). This is particularly evident on the `guassians` dataset, with the final generation resembling a "ring" with distinct points.

Across `swiss_roll` and `circles`, performance (density/coverage) improves with step count for MeanFlow, as expected. The story is flipped for `gaussians` which appears non-monotonic, i.e., 2-step (0.71/0.83) stronger than 5-step (0.67/0.73).

This is likely related to the point collapsing observation above. Using this hypothesis, we can actually **explain** the per dataset performance as a function of their ground truth distributions. Both `swiss_roll` and `circles` are connected manifolds, the data lies on a single curve, or two concentric rings. gaussians is the opposite, 8 isolated cluster modes with empty space between them.

Now let's take a step back, we have observed the behaviour (both qualitative and quantativly), but now let's unpick **why**. The answer lies in the idea of learning mean velocity. The model has to fit a single smooth function $u(z_t, r, t)$ that captures the average direction over an interval. For connected manifolds the field is continuous, nearby noise points map to nearby targets, and a smooth MLP fits this naturally. For `gaussians` the field needs *hard discontinuities* at basin boundaries (two nearby noise points routing to different clusters), and a smooth network has no way to express that. Near a boundary the model averages across the two cluster directions and predicts an "in between" velocity. The "ring of distinct points" we noted earlier is exactly this, samples landing on

the geometric average of nearby clusters because the network can't commit to one or the other.

This also explains why more steps makes it worse on gaussians. At 1 step the model is queried once for $u(z_1, 0, 1)$ and commits, the error at basin boundaries exists but only one decision is made. At 5 steps the trajectory passes through intermediate states where the field is itself ambiguous (the conditional mean velocity at points equidistant from two clusters is genuinely undefined), and the smoothing artifact gets re-evaluated at every sub-step. The multi-step refinement story only works when the underlying field is continuous.

Appendix

Table 1

stage	h	D	pred	loss	params	size	time	final loss
baseline	256	2	x	x	297218	1.19 MB	0m 47s	0.2632
baseline	256	2	x	v	297218	1.19 MB	0m 45s	0.9808
baseline	256	2	v	x	297218	1.19 MB	0m 48s	0.2637
baseline	256	2	v	v	297218	1.19 MB	0m 46s	0.9451
baseline	256	8	x	x	300296	1.21 MB	0m 49s	0.0658
baseline	256	8	x	v	300296	1.21 MB	0m 45s	0.2475
baseline	256	8	v	x	300296	1.21 MB	0m 47s	0.0666
baseline	256	8	v	v	300296	1.21 MB	0m 46s	0.2385
baseline	256	32	x	x	312608	1.26 MB	0m 48s	0.0165
baseline	256	32	x	v	312608	1.26 MB	0m 45s	0.0607
baseline	256	32	v	x	312608	1.26 MB	0m 49s	0.0185
baseline	256	32	v	v	312608	1.26 MB	0m 46s	0.0716
opt001	256	2	x	x	297218	1.19 MB	0m 45s	0.2629

stage	h	D	pred	loss	params	size	time	final loss
opt001	256	2	x	v	297218	1.19 MB	0m 47s	0.9768
opt001	256	2	v	x	297218	1.19 MB	0m 47s	0.2637
opt001	256	2	v	v	297218	1.19 MB	0m 46s	0.9451
opt001	256	8	x	x	300296	1.21 MB	0m 46s	0.0658
opt001	256	8	x	v	300296	1.21 MB	0m 47s	0.2438
opt001	256	8	v	x	300296	1.21 MB	0m 47s	0.0659
opt001	256	8	v	v	300296	1.21 MB	0m 46s	0.2357
opt001	256	32	x	x	312608	1.26 MB	0m 47s	0.0165
opt001	256	32	x	v	312608	1.26 MB	0m 49s	0.0607
opt001	256	32	v	x	312608	1.26 MB	0m 48s	0.0165
opt001	256	32	v	v	312608	1.26 MB	0m 47s	0.0587
opt002_h512	512	2	x	x	1118722	4.48 MB	0m 47s	0.2631
opt002_h512	512	2	x	v	1118722	4.48 MB	0m 47s	0.9883
opt002_h512	512	2	v	x	1118722	4.48 MB	0m 47s	0.2633
opt002_h512	512	2	v	v	1118722	4.48 MB	0m 46s	0.9444
opt002_h512	512	8	x	x	1124872	4.50 MB	0m 46s	0.0657
opt002_h512	512	8	x	v	1124872	4.50 MB	0m 47s	0.2419
opt002_h512	512	8	v	x	1124872	4.50 MB	0m 47s	0.0659
opt002_h512	512	8	v	v	1124872	4.50 MB	0m 46s	0.2349
opt002_h512	512	32	x	x	1149472	4.60 MB	0m 48s	0.0165
opt002_h512	512	32	x	v	1149472	4.60 MB	0m 49s	0.0605

stage	h	D	pred	loss	params	size	time	final loss
opt002_h512	512	32	v	x	1149472	4.60 MB	0m 49s	0.0165
opt002_h512	512	32	v	v	1149472	4.60 MB	0m 48s	0.0589
opt002_h1024	1024	2	x	x	4334594	17.34 MB	1m 13s	0.2635
opt002_h1024	1024	2	x	v	4334594	17.34 MB	1m 13s	0.9721
opt002_h1024	1024	2	v	x	4334594	17.34 MB	1m 14s	0.2636
opt002_h1024	1024	2	v	v	4334594	17.34 MB	1m 13s	0.9446
opt002_h1024	1024	8	x	x	4346888	17.39 MB	1m 13s	0.0658
opt002_h1024	1024	8	x	v	4346888	17.39 MB	1m 13s	0.2444
opt002_h1024	1024	8	v	x	4346888	17.39 MB	1m 13s	0.0659
opt002_h1024	1024	8	v	v	4346888	17.39 MB	1m 13s	0.2357
opt002_h1024	1024	32	x	x	4396064	17.59 MB	1m 15s	0.0165
opt002_h1024	1024	32	x	v	4396064	17.59 MB	1m 15s	0.0612
opt002_h1024	1024	32	v	x	4396064	17.59 MB	1m 15s	0.0165
opt002_h1024	1024	32	v	v	4396064	17.59 MB	1m 15s	0.0588

Table 2

dataset	method	steps	Density	Coverage	train
swiss_roll	FM	1	0.000	0.000	0m 48s
swiss_roll	FM	10	0.308	0.530	0m 48s
swiss_roll	FM	50	0.589	0.606	0m 48s
swiss_roll	MF	1	0.081	0.236	1m 35s
swiss_roll	MF	2	0.180	0.384	1m 35s
swiss_roll	MF	5	0.313	0.523	1m 35s
gaussians	FM	1	0.000	0.000	0m 48s
gaussians	FM	10	0.943	0.589	0m 48s
gaussians	FM	50	0.998	0.638	0m 48s

dataset	method	steps	Density	Coverage	train
gaussians	MF	1	0.583	0.707	1m 35s
gaussians	MF	2	0.712	0.832	1m 35s
gaussians	MF	5	0.670	0.727	1m 35s
circles	FM	1	0.000	0.000	0m 48s
circles	FM	10	0.816	0.873	0m 48s
circles	FM	50	0.884	0.898	0m 48s
circles	MF	1	0.532	0.742	1m 34s
circles	MF	2	0.652	0.857	1m 34s
circles	MF	5	0.679	0.877	1m 34s

MeanFlow Implementation

```python
# ---------- Model: forward pass ([src/model.py](src/model.py)) ----------
class JiM_MF(nn.Module):
    """MLP for MeanFlow: conditions on both t and the interval h."""
    def __init__(self, hidden_dim=256, D=2, t_embd=128, h_embd=128):
        super().__init__()
        in_dim = D + t_embd + h_embd
        self.layers = nn.Sequential(
            nn.Linear(in_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, D),
        )

    def forward(self, z, t, h):
        et = sinusoidal_embd(t)             # time embedding
        eh = sinusoidal_embd(h)             # interval embedding
        g  = torch.cat([z, et, eh], dim=-1)
        return self.layers(g)               # predicts x_pred (clean sample)


# ---------- Training: MeanFlow loss ([src/train.py:33](src/train.py#L33)) ----------
def meanflow_loss(model, x, fm_ratio=0.5, clamp=0.01):
    B = x.shape[0]
    # Sample (t, r) with r <= t; with prob fm_ratio collapse r=t (pure FM target)
    t  = torch.rand(B, device=x.device).clamp(min=clamp)
    r  = torch.rand(B, device=x.device) * t
    is_fm = torch.rand(B, device=x.device) < fm_ratio
    r  = torch.where(is_fm, t, r)
    h  = t - r
```

```
    eps = torch.randn_like(x)
    z_t = (1 - t[:, None]) * x + t[:, None] * eps      # interpolant
    v   = eps - x                                       # instantaneous velocity

    # Express average velocity u via x-prediction so JVP differentiates the whole map
    def u_func(z_, t_, h_):
        x_pred = model(z_, t_, h_)
        return (z_ - x_pred) / t_[:, None]

    tgnt      = (v, torch.ones_like(t), torch.ones_like(h))
    u, dudt   = torch.func.jvp(u_func, (z_t, t, h), tgnt)

    # MeanFlow target: u_tgt = v - h * du/dt   (stop-grad on target)
    u_tgt_mf = (v - h[:, None] * dudt).detach()
    u_tgt    = torch.where(is_fm[:, None], v, u_tgt_mf)  # FM fallback when h=0
    return F.mse_loss(u, u_tgt)


# ---------- Denoising: few-step sampler ([src/denoiser.py:46](src/denoiser.py#L46)) ------
class Denoiser_MF:
    def __init__(self, model, steps=5, D=2):
        self.model, self.steps, self.D = model, steps, D

    @torch.no_grad()
    def generate(self, bsz, device):
        z = torch.randn(bsz, self.D, device=device)         # z_1 ~ N(0, I)
        interval = 1.0 / self.steps
        h = torch.full((bsz,), interval, device=device)

        for i in range(self.steps):
            t_val = 1.0 - i * interval                      # 1.0, 0.8, 0.6, ...
            t     = torch.full((bsz,), t_val, device=device)
            x_pred = self.model(z, t, h)                    # x-prediction

            # move z toward x_pred
            alpha  = (h / t)[:, None]
            z      = (1 - alpha) * z + alpha * x_pred
        return z
```

References

[1] Naeem et al., “Reliable Fidelity and Diversity Metrics for Generative Models”, ICML 2020 (arXiv:2002.09797).

[2] Li et al. “Back to Basics: Let Denoising Generative Models Denoise”, 2026

(arXiv:2511.13720v2).

[3] Zheng et al. “Diffusion Transformers with Representational Autoencoders”, 2025 (arXiv:2510.11690v1).

[4] Stability AI. “Scaling Rectified Flow Transformers for High-Resolution Image Synthesis”, 2024 (arXiv:2403.03206v).

[5] Black Forest Labs. “FLUX.1 Kontext: Flow Matching for In-Context Image Generation and Editing in Latent Space”, 2025 (arXiv:2506.15742).

[6] Geng et al. “Mean Flows for One-step Generative Modeling”, 2025 (arXiv:2505:13447v1).